[SESSION 08] $ ./SQL

آنلاین •

رایگان

دورهٔ رایگان مهندسی داده و هوش مصنوعی

جلسهٔ دوم

SQL، قسمت دوم:
از کوئری تا پایپ‌لاین

Window Function، شش تله‌ی رایج، الگوهای کلاسیک، SQL در پایپ‌لاین و SQL in Scale؛ همه روی کوئری.

تاریخ

یکشنبه 10 آبان

ساعت

۲۰:۰۰



ارائه‌دهنده

کوثر اصغری

مدرس دوره

t.me/data_ai_engineering

من کی‌ام؟

EDUCATION

تحصیلات

فارغ‌التحصیل کارشناسی مهندسی هوافضا دانشگاه صنعتی شریف

EXPERIENCE

تجربه

سرپرست تیم مدیریت داده در حوزه کارگزاری بین‌المللی

مهندس تحلیل داده و مشاور داده کیمیاگران نفت و LABELL

تحلیلگر داده و محصول یکتانت، پرس‌لاین، Nexu و eVTOL.ae

امروز چی می‌گیم؟

01 · 12'

مرور و Window

تمرین کلاسی مرور، فکر کردن با مجموعه‌ها و Window Function روی داده‌ی واقعی

02 · 23'

تله‌ها

شش تله‌ی اصلی، تله‌های عددی و SQL Injection

03 · 16'

الگوهای کلاسیک

Top-N، حذف تکراری، رشد ماهانه، قیف و point-in-time

04 · 10'

SQL در پایپ‌لاین

fact و dimension، لود تکرارپذیر و افزایشی، تست کیفیت

05 · 13'

SQL in Scale

سطری یا ستونی، ایندکس، EXPLAIN و روش بهینه‌سازی

06 · 3'

جمع‌بندی

منابع و تمرین بعد از کلاس

تمرین کلاسی ۱: مرور جلسهٔ قبل

```
-- Class exercise 1
--
-- For each order status in 2026,
-- count the orders.
--
-- Keep only statuses with more
-- than 2000 orders.
--
-- Sort from most to fewest.
```

TABLE

جدول orders؛ ستون‌های status و order_at.

SYNTAX

WHERE، GROUP BY، HAVING، ORDER BY و COUNT.

RANGE

سال ۲۰۲۶ یعنی از ابتدای ۲۰۲۶ تا قبل از ابتدای ۲۰۲۷.

TIME

سه دقیقه.

جواب تمرین کلاسی ۱

```sql
SELECT status, COUNT(*) AS orders
FROM orders
WHERE order_at ⩾ '2026-01-01'
  AND order_at <  '2027-01-01'
GROUP BY status
HAVING COUNT(*) > 2000
ORDER BY orders DESC;

-- status    | orders
-- paid      | 27327
-- cancelled | 2759
-- pending   | 2449
```

WHERE
ردیف‌ها رو قبل از گروه‌بندی فیلتر می‌کنه: فقط سفارش‌های ۲۰۲۶.

HAVING
گروه‌ها رو بعد از گروه‌بندی فیلتر می‌کنه؛ شرط روی COUNT فقط اینجا می‌شه.

REFUNDED
۱۷۴۷ سفارش refunded در ۲۰۲۶ داشتیم، کمتر از ۲۰۰۰؛ برای همین HAVING حذفش کرد.

HALF-OPEN RANGE
بزرگ‌تر-مساوی شروع و کوچک‌تر از پایان؛ دلیلش در تله‌ی ۵ همین جلسه.

بخش ۱۰

حلقه نزن،
Window بزن

SQL با مجموعه‌ها کار می‌کنه، نه ردیف به ردیف.

به‌جای حلقه، یک کوئری

N+1 · SLOW

یک کوئری برای هر مشتری

۵۰۰۰ مشتری یعنی ۵۰۰۰ بار رفت‌وبرگشت به دیتابیس.

```
for cid in customer_ids:
    cur.execute(
        "SELECT SUM(order_amount) "
        "FROM orders "
        "WHERE customer_id = %s", (cid,))
```

SET-BASED · FAST

یک کوئری برای همه

دیتابیس کل کار رو یک‌جا و بهینه انجام می‌ده.

```
SELECT customer_id,
       SUM(order_amount)
FROM orders
GROUP BY customer_id;
```

همین منطق برای UPDATE هم هست. به‌جای حلقه‌ای که تک‌تک ردیف‌ها رو آپدیت کنه، یک UPDATE با شرط بنویسید؛ دیتابیس برای کار روی مجموعه ساخته شده.

Window Function: سه تابع، یک پنجره

```
SELECT order_id, order_amount,
       ROW_NUMBER() OVER w AS nth_order,
       LAG(order_amount) OVER w AS prev_amount,
       SUM(order_amount) OVER w AS running_total
FROM orders
WHERE customer_id = 1
WINDOW w AS (PARTITION BY customer_id ORDER BY order_at)
ORDER BY order_at
LIMIT 4;

-- order_id | amount | nth | prev   | running
-- 13780    | 106.52 | 1   | NULL   | 106.52
-- 12858    | 367.61 | 2   | 106.52 | 474.13
-- 49319    | 102.78 | 3   | 367.61 | 576.91
-- 25482    | 229.38 | 4   | 102.78 | 806.29
```

ROW_NUMBER

سفارش چندم این مشتریه؟

LAG

مبلغ سفارش قبلی؛ اولین سفارش قبلی نداره، پس NULL.

SUM() OVER

جمع خرید تا همین سفارش.

LEAD

برعکس LAG: مقدار ردیف بعدی. آخرین سفارش NULL می‌گیره.

OVER بدون ORDER BY: کل گروه

```sql
SELECT order_id, order_amount,
       SUM(order_amount) OVER (PARTITION BY customer_id)
         AS customer_total,
       ROUND(100 * order_amount
             / SUM(order_amount) OVER (PARTITION BY customer_id),
             1) AS pct_of_customer
FROM orders
WHERE customer_id = 1
ORDER BY order_at
LIMIT 4;

-- order_id | amount | customer_total | pct
-- 13780    | 106.52 | 11119.72       | 1.0
-- 12858    | 367.61 | 11119.72       | 3.3
-- 49319    | 102.78 | 11119.72       | 0.9
```

NO ORDER BY

بدون ORDER BY داخل OVER، پنجره کل گروهه، نه «تا اینجا».

SAME VALUE

customer_total برای همه‌ی ردیف‌های این مشتری یکیه.

WHY

سهم هر ردیف از کل گروه، بدون GROUP BY و بدون JOIN.

VS Q03

فرق این کوئری با Q03 فقط نبودن ORDER BY داخل OVER هست.

فیلتر روی Window Function

WRONG

فیلتر در همون کوئری

ستون rn هنوز وجود نداره.

```
SELECT customer_id, order_id,
       ROW_NUMBER() OVER (PARTITION BY customer_id
                          ORDER BY order_at) AS rn
FROM orders
WHERE rn = 1;
-- ERROR: column "rn" does not exist
```

RIGHT

اول بساز، بعد فیلتر کن

داخل CTE بساز و بیرونش فیلتر کن.

```
WITH numbered AS (
  SELECT customer_id,
    ROW_NUMBER() OVER (PARTITION BY customer_id
                       ORDER BY order_at) AS rn
  FROM orders)
SELECT COUNT(*) FROM numbered WHERE rn = 1; -- 4,250
```

دلیل: ترتیب اجرای جلسه‌ی قبل: WHERE قبل از SELECT اجرا می‌شه و Window Function توی SELECT ساخته می‌شه. این الگو پایه‌ی Top-N و حذف تکراری‌هاست.

بخش ۰۲

شش تله‌ای که جواب غلط می‌دن

کوئری خطا نمی‌ده، ولی عدد غلطه؛ خطرناک‌ترین نوع باگ.

تله ۱: Fan-out، وقتی JOIN عددها رو چند برابر می‌کنه

WRONG

سه جدول رو مستقیم JOIN کن

سفارش سه‌قلمی با دو قسط، شش بار شمرده می‌شه.

```
SELECT SUM(o.order_amount)
FROM orders o
JOIN order_items i USING (order_id)
JOIN payments p    USING (order_id)
WHERE o.status = 'paid';
-- 18,366,226
```

RIGHT

اول خلاصه کن، بعد JOIN

پرداخت‌ها رو اول تا گرین سفارش جمع بزن.

```
WITH paid AS (SELECT order_id, SUM(amount) AS paid
              FROM payments GROUP BY order_id)
SELECT SUM(o.order_amount)
FROM orders o JOIN paid p USING (order_id)
WHERE o.status = 'paid';
-- 5,770,297
```

دلیل: هر سفارش در «تعداد قلم × تعداد قسط» ضرب می‌شه. چک سریع: بعد از هر JOIN، تعداد ردیف‌ها رو با تعداد سفارش‌های یکتا مقایسه کن؛ اگه برابر نیستن، fan-out داری.

تله ۲: LEFT JOIN که بی‌صدا INNER می‌شه

WRONG

شرط جدول دوم در WHERE

مشتری‌هایی که سفارش paid ندارن، حذف می‌شن.

```sql
SELECT c.customer_id, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'paid'
GROUP BY c.customer_id;
-- 4,250 rows
```

RIGHT

شرط رو ببر توی ON

همه‌ی مشتری‌ها می‌مونن؛ بی‌سفارش‌ها با صفر.

```sql
SELECT c.customer_id, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
                  AND o.status = 'paid'
GROUP BY c.customer_id;
-- 5,000 rows
```

دلیل: برای مشتری بی‌سفارش، ستون status خالی (NULL) میاد و شرط WHERE براش درست نمی‌شه؛ پس اون ردیف حذف می‌شه و LEFT JOIN عملاً INNER می‌شه.

تله ۳: NULL یعنی «نمی‌دونیم»، نه «خالی»

WRONG

مقایسه‌ی معمولی با NULL

مشتری‌هایی که کشورشون نامعلومه، جا می‌مونن.

```
SELECT COUNT(*) FROM customers
WHERE country <> 'Iran';     -- 1,473

SELECT COUNT(*) FROM customers
WHERE city = NULL;           -- 0
```

RIGHT

صریح با NULL برخورد کن

NULL رو جدا در نظر بگیر.

```
SELECT COUNT(*) FROM customers
WHERE country IS DISTINCT FROM 'Iran';  -- 1,621

SELECT COUNT(*) FROM customers
WHERE city IS NULL;          -- 409
```

دلیل: هر مقایسه با NULL جوابش «نامعلوم»ه و WHERE فقط ردیف‌های «درست» رو نگه می‌داره. حواستون باشه COUNT(city) و AVG هم NULLها رو نادیده می‌گیرن.

تله ۴: NOT IN و یک NULL که همه‌چیز رو صفر می‌کنه

WRONG

NOT IN با زیرکوئری

شهرهایی که هیچ مشتری ترکیه‌ای اونجا نیست.

```
SELECT COUNT(*) FROM customers
WHERE city NOT IN (
  SELECT city FROM customers
  WHERE country = 'Turkey');
-- 0
```

RIGHT

NOT EXISTS

با NULL درست رفتار می‌کنه.

```
SELECT COUNT(*) FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM customers t
  WHERE t.country = 'Turkey'
    AND t.city = c.city);
-- 4,543
```

دلیل: ۴۱ مشتری ترکیه شهر ندارن، پس توی لیست NULL هست. دیتابیس نمی‌تونه مطمئن بگه شهری «توی لیست نیست»، پس هیچ ردیفی برنمی‌گردونه.

تله ۵: بازه‌ی زمانی؛ روز آخر کجا رفت؟

WRONG

BETWEEN با تاریخ

تاریخ بدون ساعت یعنی نیمه‌شبِ شروع اون روز.

```sql
SELECT SUM(order_amount)
FROM orders
WHERE order_at BETWEEN '2026-09-01'
                   AND '2026-09-30';
-- 849,316.84
```

RIGHT

بازه‌ی نیمه‌باز

از شروع، تا قبل از روز بعد.

```sql
SELECT SUM(order_amount)
FROM orders
WHERE order_at ⩾ '2026-09-01'
  AND order_at <  '2026-10-01';
-- 876,572.65
```

حتی ساعت ۲۳:۵۹:۵۹ هم کافی نیست: سفارش ۷۷۷ نیم ثانیه بعدش ثبت شده و جا می‌مونه. همیشه بزرگ‌تر-مساوی شروع، و کوچک‌تر از پایان.

تله ۶: UPDATE و DELETE؛ اول ببین، بعد تغییر بده

WRONG

یک WHERE جا افتاده

همه‌ی ۵۰ هزار سفارش لغو می‌شن.

```
UPDATE orders
SET status = 'cancelled';
-- UPDATE 50000
```

RIGHT

اول SELECT، بعد داخل تراکنش

تعداد رو ببین، اگه درست بود COMMIT کن.

```
BEGIN;
SELECT COUNT(*) FROM orders
WHERE status = 'pending'
  AND order_at < '2025-02-01';   -- 47
UPDATE orders SET status = 'cancelled' WHERE ...;
COMMIT;   -- or ROLLBACK
```

عادت خوب: WHERE رو اول با SELECT امتحان کن، بعد همون رو به UPDATE بده. بعد از COMMIT دیگه برگشتی نیست؛ فقط بکاپ.

تله‌های عددی

```sql
-- integer division
SELECT SUM(quantity) / COUNT(*)
FROM order_items;                          -- 1
SELECT SUM(quantity)::numeric / COUNT(*)
FROM order_items;                          -- 1.61

-- FLOAT is approximate
SELECT SUM(0.1::float)
FROM generate_series(1, 10);  -- 0.9999999999999999

-- division by zero
SELECT 10 / 0;                -- ERROR
SELECT 10 / NULLIF(0, 0);     -- NULL
```

INTEGER
تقسیم دو عدد صحیح، اعشار رو دور می‌ریزه. یکی رو به numeric تبدیل کن.

MONEY
برای پول همیشه NUMERIC، هیچ‌وقت FLOAT.

AVG OF AVGS
میانگین میانگین‌ها به گروه کوچیک و بزرگ وزن برابر می‌ده؛ SUM تقسیم بر COUNT.

LIMIT · DISTINCT
LIMIT بدون ORDER BY ترتیب تضمین‌شده نداره.
DISTINCT جای پیدا کردن fan-out نیست.

SQL Injection: ورودی کاربر، کد نیست

WRONG

چسباندن رشته

یک تکه SQL به‌جای ایمیل، کل جدول رو بیرون می‌کشه.

```
# email = "x' OR '1'='1"
q = ("SELECT * FROM customers "
     f"WHERE email = '{email}'")
cur.execute(q)
```

RIGHT

کوئری پارامتری

دیتابیس ورودی رو فقط «داده» می‌بینه، نه بخشی از کوئری.

```
cur.execute(
    "SELECT * FROM customers "
    "WHERE email = %s",
    (email,))
```

برای مهندس AI: ایجنتی که خودش SQL می‌نویسه (text-to-SQL) رو با کاربری وصل کنید که فقط دسترسی خواندن داره.

قبل از تحویل هر کوئری

- هر ردیف خروجی یعنی چی؟ (گرین)
- بعد از JOIN، تعداد ردیف‌ها با تعداد کلیدهای یکتا می‌خونه؟
- NULLها کجا رفتن؟ حذف شدن یا جایی شمرده نشدن؟
- بازه‌های زمانی نیمه‌بازن؟ (>= شروع و < پایان)
- عدد نهایی رو با یک منبع دیگه یا یک حساب سرانگشتی چک کردم؟

بخش ۰۳

الگوهای
کلاسیک مسائل

سوال‌هایی که در هر شرکتی بارها تکرار می‌شن.

شش الگوی پرتکرار

TOP-N PER GROUP

چندتای اول هر گروه

«سه محصول پرفروش هر دسته؟»

```
DENSE_RANK() OVER (...)
```

DEDUPLICATION

آخرین نسخه

«آخرین وضعیت هر سفارش؟»

```
ROW_NUMBER() ... = 1
```

RUNNING TOTAL

جمع تا اینجا

«درآمد تجمعی از اول سال؟»

```
SUM() OVER (ORDER BY ...)
```

PERIOD OVER PERIOD

مقایسه با قبل

«رشد این ماه نسبت به ماه قبل؟»

```
LAG() + SUM() OVER
```

FUNNEL · PIVOT

شمارش شرطی

«چند نفر به هر مرحله رسیدن؟»

```
COUNT(*) FILTER (WHERE ...)
```

ANTI-JOIN

چیزی که نیست

«کی هیچ‌وقت خرید نکرده؟»

```
WHERE NOT EXISTS (...)
```

مثال: سه محصول پرفروش هر دسته

```sql
WITH product_revenue AS (
  SELECT p.category, p.product_name,
         SUM(i.quantity * i.unit_price) AS revenue
  FROM order_items i
  JOIN products p USING (product_id)
  GROUP BY p.category, p.product_name
)
SELECT *
FROM (
  SELECT *,
         DENSE_RANK() OVER (
           PARTITION BY category
           ORDER BY revenue DESC) AS rnk
  FROM product_revenue
) t
WHERE rnk ⩽ 3;
```

STEP 1

درآمد هر محصول با GROUP BY.

STEP 2

داخل هر دسته رتبه بده؛ رتبه برای هر دسته از ۱ شروع می‌شه.

STEP 3

بیرون از زیرکوئری فیلتر کن؛ همون الگوی Q05.

RANK FUNCTIONS

در تساوی: ROW_NUMBER به هر ردیف عدد جدا، RANK و DENSE_RANK به مساوی‌ها عدد یکسان می‌دن.

تمرین کلاسی ۲: دو شهر پرمشتری هر کشور

```
-- Class exercise 2
--
-- For each country, find the two
-- cities with the most customers.
--
-- Ignore customers whose country
-- or city is NULL.
--
-- Show country, city, customers
-- and the rank.
```

TABLE

جدول customers؛ ستون‌های country و city.

STEP 1

تعداد مشتری هر شهر در هر کشور با GROUP BY.

STEP 2

رتبه‌بندی داخل هر کشور با ROW_NUMBER و PARTITION BY.

STEP 3

فیلتر رتبه بیرون از زیرکوئری یا CTE. زمان: دو دقیقه.

جواب تمرین کلاسی ۲

```sql
WITH city_counts AS (
  SELECT country, city, COUNT(*) AS customers
  FROM customers
  WHERE country IS NOT NULL AND city IS NOT NULL
  GROUP BY country, city
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY country
                    ORDER BY customers DESC) AS rn
  FROM city_counts
)
SELECT * FROM ranked WHERE rn ⩽ 2 ORDER BY country, rn;

-- Canada | Toronto  | 418  | 1
-- Iran   | Tehran   | 1370 | 1
-- Iran   | Mashhad  | 457  | 2
-- Turkey | Istanbul | 450  | 1
-- UAE    | Dubai    | 472  | 1
```

SAME PATTERN

همون الگوی Top-N: اول خلاصه، بعد رتبه، بعد فیلتر.

ONE ROW

کانادا، ترکیه و امارات توی این دیتاست فقط یک شهر دارن؛ Top-N یعنی «حداکثر» N تا.

NULL

بدون شرط IS NOT NULL، شهر خالی هم به‌عنوان یک «شهر» رتبه می‌گرفت.

ROW_NUMBER

اگه دو شهر تعداد برابر داشتن، فقط یکی می‌اومد؛ برای آوردن هر دو DENSE_RANK.

حذف تکراری: آخرین نسخه‌ی هر سفارش

```sql
SELECT order_id, status, updated_at, batch_id
FROM orders_staging
WHERE order_id = 260
ORDER BY updated_at;
-- 260 | pending | 03:01:32 | 1
-- 260 | paid    | 05:01:32 | 2
-- 260 | paid    | 05:01:32 | 2   ← duplicate

SELECT COUNT(*)
FROM (
  SELECT *, ROW_NUMBER() OVER (
              PARTITION BY order_id
              ORDER BY updated_at DESC, batch_id DESC) AS rn
  FROM orders_staging
) t
WHERE rn = 1;                -- 11,633 rows → 6,185
```

RAW FEED

فید خام از هر سفارش چند نسخه داره، حتی ردیف کاملاً تکراری.

DESC

جدیدترین نسخه شماره‌ی ۱ می‌گیره.

TIE-BREAK

batch_id دوم برای وقتیه که زمان دو نسخه یکیه.

NEXT

خروجی همین کوئری رو در بخش پایپ‌لاین لود می‌کنیم.

رشد ماهانه و جمع تجمعی

```sql
WITH monthly AS (
  SELECT date_trunc('month', order_at)::date AS month,
         SUM(order_amount) AS revenue
  FROM orders
  WHERE status = 'paid' AND order_at ⩾ '2026-01-01'
  GROUP BY 1
)
SELECT month, revenue,
       ROUND(100 * (revenue / LAG(revenue) OVER (ORDER BY month)
                    - 1), 1) AS growth_pct,
       SUM(revenue) OVER (ORDER BY month) AS running_total
FROM monthly
ORDER BY month;

-- month      | revenue   | growth_pct | running_total
-- 2026-01-01 | 271146.11 |            | 271146.11
-- 2026-02-01 | 261193.84 | -3.7       | 532339.95
-- 2026-03-01 | 330873.47 | 26.7       | 863213.42
```

CTE FIRST

اول یک ردیف برای هر ماه؛ بعد Window روی همون.

LAG

درآمد ماه قبل؛ ژانویه ماه قبل نداره، پس رشدش خالیه.

RUNNING TOTAL

SUM با ORDER BY یعنی جمع از اول تا همین ماه.

TWO PATTERNS

Period over Period و Running Total در یک کوئری.

مثال: قیف خرید سپتامبر

```sql
SELECT
  COUNT(DISTINCT customer_id)
    FILTER (WHERE event_type = 'page_view')
    AS viewed,
  COUNT(DISTINCT customer_id)
    FILTER (WHERE event_type = 'add_to_cart')
    AS added,
  COUNT(DISTINCT customer_id)
    FILTER (WHERE event_type = 'purchase')
    AS bought
FROM events
WHERE event_at ⩾ '2026-09-01'
  AND event_at <  '2026-10-01';
```

IDEA

یک بار جدول رو بخون و چند شمارش شرطی با هم بگیر؛ نه چهار کوئری جدا.

FILTER

فقط ردیف‌هایی رو می‌شمره که شرط براشون درسته.

RESULT

۵۰۰۰ نفر دیدن، ۴۶۳۵ نفر به سبد اضافه کردن، ۳۶۷۶ نفر خریدن.

OTHER DATABASES

اگه دیتابیسی FILTER نداشت، همین کار رو با CASE WHEN داخل COUNT انجام بده.

Point-in-time: داده در همون لحظه

```sql
SELECT o.order_id, o.order_at::date,
       s.segment   AS at_order_time,
       cur.segment AS current
FROM orders o
JOIN customer_segments s
  ON s.customer_id = o.customer_id
 AND s.valid_from ⩽ o.order_at
 AND (s.valid_to IS NULL OR o.order_at < s.valid_to)
JOIN customer_segments cur
  ON cur.customer_id = o.customer_id
 AND cur.valid_to IS NULL
WHERE o.customer_id = 1
ORDER BY o.order_at;

-- 13780 | 2025-01-03 | bronze | gold
-- 12858 | 2025-01-05 | bronze | gold
```

SCD TYPE 2

هر ردیف یک بازه‌ی زمانیه: از valid_from تا valid_to.

AS-OF JOIN

سگمنتی رو بگیر که لحظه‌ی سفارش معتبر بوده.

LEAKAGE

در کل داده، ۱۲,۲۴۸ از ۵۰ هزار سفارش با سگمنت فعلی برچسب غلط می‌گیرن.

AI

مدلی که آینده رو دیده، توی تست عالیه و توی واقعیت نه.

برای بعد: سه الگوی چالشی

GAPS & ISLANDS

روزهای پشت‌سرهم

طولانی‌ترین زنجیره‌ی روزهای فعال هر مشتری.

`date - ROW_NUMBER()`

SESSIONIZATION

تقسیم به session

فاصله‌ی بیشتر از ۳۰ دقیقه بین دو کلیک یعنی session جدید.

`LAG() + SUM() OVER`

COHORT

مشتری‌های هم‌ورودی

از کسایی که اولین خریدشون یک ماه بوده، چند نفر ماه‌های بعد برگشتن؟

`MIN() + date_trunc`

در تمرین‌های جلسه‌ی دوم: شماره‌ی ۱۱ تا ۱۳، با جواب کامل. همه‌شون با همین ابزارهای امروز حل می‌شن.

بخش ۰۴

SQL در
پایپ‌لاین داده

کوئری‌ای که هر شب خودکار اجرا می‌شه، باید قابل اعتماد باشه.

مدل‌سازی: fact، dimension و گرین

FACT

جدول رویداد

هر ردیف یک اتفاق: سفارش، پرداخت. مدام زیاد می‌شه.

orders · payments · events

DIMENSION

جدول توصیف

هر ردیف یک «چیز»: مشتری، محصول. کم عوض می‌شه.

customers · products

GRAIN

یک ردیف یعنی چی؟

اولین تصمیم هر جدول؛ بدونش JOIN عددها رو چند برابر می‌کنه.

1 row = 1 order

به این چیدمان می‌گن Star Schema: جدول fact در وسط و dimensionها دورش؛ با کلیدها به هم وصل می‌شن و با JOIN کنار هم میان.

اجرای دوباره نباید داده‌ی تکراری بسازه

WRONG

INSERT ساده

اگه job دو بار اجرا بشه (و می‌شه!)، هر سفارش دو بار ثبت می‌شه.

```
INSERT INTO orders_clean
SELECT * FROM latest_orders;
```

RIGHT · UPSERT

اگه بود آپدیت، اگه نبود اضافه

ده بار هم اجرا بشه، نتیجه همونه.

```
INSERT INTO orders_clean
SELECT * FROM latest_orders
ON CONFLICT (order_id) DO UPDATE
SET status     = EXCLUDED.status,
    updated_at = EXCLUDED.updated_at;
```

اسمش idempotency هست: یعنی اجرای دوباره، نتیجه رو عوض نکنه. هر job در پایپ‌لاین بالاخره یک روز دوباره اجرا می‌شه.

لود افزایشی و داده‌ی دیررس

```sql
-- full load: everything, every night
SELECT COUNT(*) FROM orders_staging;      -- 11,633

-- incremental: only changes since the last run
SELECT COUNT(*) FROM orders_staging
WHERE updated_at > TIMESTAMP '2026-09-28'; -- 1,140

-- incremental + 2-day lookback for late data
SELECT COUNT(*) FROM orders_staging
WHERE updated_at > TIMESTAMP '2026-09-28'
                 - INTERVAL '2 days';    -- 1,899
```

FULL LOAD

ساده و مطمئن، ولی با بزرگ شدن داده هر شب کندتر.

WATERMARK

زمان آخرین اجرا؛ فقط چیزهایی که بعدش عوض شدن.

LOOKBACK

بعضی ردیف‌ها دیر می‌رسن؛ یک بازه‌ی عقب‌تر رو هم دوباره بخون.

WHY UPSERT

ردیف‌های دوباره‌خونده، بدون UPSERT تکراری می‌شن.

تست کیفیت داده: باید صفر ردیف بده

```sql
-- unique key
SELECT order_id FROM orders_clean
GROUP BY order_id HAVING COUNT(*) > 1;          -- 0 rows
-- same test on the raw feed
SELECT order_id FROM orders_staging
GROUP BY order_id HAVING COUNT(*) > 1;          -- 5,213 rows

-- not null
SELECT * FROM orders_clean
WHERE customer_id IS NULL OR order_at IS NULL;  -- 0 rows

-- referential integrity
SELECT * FROM orders_clean oc
WHERE NOT EXISTS (
  SELECT 1 FROM customers c
  WHERE c.customer_id = oc.customer_id);        -- 0 rows

-- reconcile with the source
SELECT * FROM orders_clean oc JOIN orders o USING (order_id)
WHERE oc.status IS DISTINCT FROM o.status;      -- 0 rows
```

IDEA

هر تست یک کوئریه که ردیف‌های خراب رو پیدا می‌کنه.

PASS / FAIL

صفر ردیف یعنی پاس. فید خام تست یکتایی رو رد می‌کنه.

RECONCILE

مقایسه با منبع؛ مهم‌ترین تست برای اعتماد به عدد.

AUTOMATE

dbt همین تست‌ها رو بعد از هر لود خودکار اجرا می‌کنه.

بخش ۰۵

SQL in Scale

وقتی داده میلیونی می‌شه، چطور کوئری سریع بنویسیم؟

دیتابیس سطری و ستونی

ROW STORE · PostgreSQL · MySQL

سطری: ردیف‌ها کنار هم

هر ردیف کامل یک‌جا ذخیره می‌شه. خوندن یا نوشتن یک سفارش کامل سریعه.

```
[1, Sara, Tehran, 250]
[2, Ali,  Shiraz, 120]
[3, Sara, Tehran,  90]
```

COLUMN STORE · ClickHouse · BigQuery

ستونی: ستون‌ها کنار هم

هر ستون جدا ذخیره می‌شه. برای جمع زدن amount، بقیه‌ی ستون‌ها اصلاً خونده نمی‌شن.

```
id:     [1, 2, 3]
name:   [Sara, Ali, Sara]
city:   [Tehran, Shiraz, Tehran]
amount: [250, 120, 90]
```

Q22 روی هر دو دیتابیس: تعداد page_view هر دستگاه. PostgreSQL کل ردیف‌ها رو می‌خونه؛ ClickHouse فقط ستون‌های event_type و device رو.

بر اساس چی بهینه کنیم؟

	سطری (OLTP)	ستونی (OLAP)
خوبه برای	ثبت و خوندن یک سفارش	جمع و میانگین روی میلیون‌ها ردیف
گنده برای	اسکن کل جدول برای یک ستون	آپدیت و حذف تک‌ردیف
هزینه از کجا میاد؟	تعداد ردیف‌هایی که لمس می‌شه	تعداد ستون‌ها و پارتیشن‌هایی که خونده می‌شه
ابزار اصلی بهینه‌سازی	ایندکس	کلید مرتب‌سازی و پارتیشن
`SELECT *`	بد	خیلی بد

قبل از بهینه‌سازی بپرس: این موتور داده رو چطور می‌خونه؟

ایندکس: مثل فهرست آخر کتاب

BEFORE

بدون ایندکس

برای پیدا کردن رویدادهای مشتری ۴۲، هر دو میلیون ردیف خونده می‌شه.

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM events
WHERE customer_id = 42;
-- Parallel Seq Scan on events
-- Execution Time: ~170 ms
```

AFTER

با ایندکس

مستقیم سراغ همون ۳۸۶ ردیف می‌ره. هزینه‌اش: فضای بیشتر و نوشتن کندتر.

```
CREATE INDEX idx_events_customer
  ON events (customer_id);
-- Index Only Scan
-- Execution Time: ~0.1 ms
```

خوندن EXPLAIN

```
HashAggregate
  Group Key: c.city
  →  Hash Join
        Hash Cond: (o.customer_id = c.customer_id)
        →  Hash Join
              Hash Cond: (p.order_id = o.order_id)
              →  Seq Scan on payments p
              →  Hash
                    →  Seq Scan on orders o
        →  Hash
              →  Seq Scan on customers c
```

READ

از داخلی‌ترین خط شروع کن؛ اول اون اجرا می‌شه.

SCAN

Seq Scan یعنی کل جدول. روی جدول بزرگ و فیلتر محدود، احتمالاً ایندکس لازمه.

JOIN

Hash Join برای جدول‌های بزرگ خوبه. Nested Loop روی دو جدول بزرگ معمولاً یعنی دردسر.

ROWS

اگه تعداد ردیف تخمینی با واقعی خیلی فرق داره، آمار جدول قدیمیه: ANALYZE.

تابع روی ستون، ایندکس رو بی‌اثر می‌کنه

SLOW

تابع روی ستون

تابع روی هر دو میلیون ردیف اجرا می‌شه.

```
CREATE INDEX idx_events_event_at
  ON events (event_at);
EXPLAIN ANALYZE
SELECT COUNT(*) FROM events
WHERE event_at::date = '2026-09-15';
-- Seq Scan, ~200 ms
```

FAST

بازه‌ی نیمه‌باز

همون جواب، با استفاده از ایندکس.

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM events
WHERE event_at ⩾ '2026-09-15'
  AND event_at <  '2026-09-16';
-- Index Only Scan, under 1 ms
```

همون بازه‌ی تله‌ی پنجم: هم درست‌تره، هم سریع‌تر. بقیه‌ی قانون‌های کوئری سریع در اسلاید بعد.

چهار قانون کوئری سریع

ONLY WHAT YOU NEED

فقط ستون‌های لازم

SELECT * در دیتابیس ستونی یعنی خوندن همه‌ی ستون‌ها.

```
SELECT city, order_amount
```

NO FUNCTION ON COLUMN

تابع روی ستون نذار

event_at::date جلوی استفاده از ایندکس رو می‌گیره؛ بازه بنویس.

```
event_at ⩾ '2026-09-15' AND event_at < '2026-09-16'
```

SHRINK FIRST

قبل از JOIN کوچیک کن

اول فیلتر و GROUP BY، بعد JOIN. هم سریع‌تر، هم بدون fan-out.

```
WITH paid AS (... GROUP BY order_id)
```

CHEAPER OPS

عملیات ارزون‌تر

UNION ALL به‌جای UNION، NOT EXISTS به‌جای NOT IN.

```
UNION ALL · EXISTS
```

کوئری کند رو چطور درست کنیم؟



هر بار فقط یک تغییر، و همیشه ثابت کن جواب عوض نشده:

```
(new EXCEPT old) UNION ALL (old EXCEPT new)   -- 0 rows
```

کوئری سریعی که جواب غلط می‌ده، از کوئری کند بدتره.

منابع پیشنهادی

START HERE

W3Schools SQL

شروع سریع، با ویرایشگر داخل صفحه

w3schools.com/sql

PRACTICE

LeetCode SQL 50

۵۰ سوال دسته‌بندی‌شده، شبیه مصاحبه

leetcode.com

MISTAKES

SQL Antipatterns

اشتباه‌های رایج طراحی و کوئری

Bill Karwin

PERFORMANCE

Use The Index, Luke!

ایندکس و پرفورمنس؛ رایگان و آنلاین

Markus Winand

MODERN SQL

modern-sql.com

Window Function، CTE و قابلیت‌های جدید

Markus Winand

MODELING

Data Warehouse Toolkit

مدل‌سازی ابعادی؛ fact و dimension

Ralph Kimball

تمرین بعد از کلاس

PRACTICE 80

۸۰ تمرین سینتکس

اگه تموم نشده، اول این‌ها. هر تمرین لینک سینتکس و عدد «برای چک» داره.

sql-practice-80-part1

TRAPS

تمرین تله‌ها

تمرین‌های ۷ تا ۱۲ جلسه‌ی اول، که درباره‌ی همین شش تله و Window Function هستن.

exercises/part1.md

SESSION 2

تمرین‌های امروز

تله‌های عددی، الگوها، لود تکرارپذیر، تست کیفیت و بخش Scale.
تمرین‌های ۱۱ تا ۱۵ چالشی و اختیاری‌ان.

exercises/part2.md

[END] $ exit 0

پایان جلسه

سوال؟

تمرین‌ها: sql-practice-80-part1 · exercises/part1.md · exercises/part2.md

عضویت در کانال تلگرام دوره

t.me/data_ai_engineering